



Hanzo ML and Hanzo Engine: A Unified Multi-Backend, Multi-Modal Inference Stack

Zach Kelling

Hanzo AI

2026

Abstract

We present Hanzo ML and Hanzo Engine, a single Rust inference stack that runs the same model across five compute backends (CUDA, ROCm, Metal, Vulkan, CPU), serves six modalities (text, vision, audio, speech, image generation, embeddings), and carries the entire GGUF quantization zoo through one unified compute core. The thesis is structural: one templated quantization core, one model loader, one set of pipelines, with each axis (weight format, activation dtype, dense-versus-mixture-of-experts, scalar-versus-dp4a) orthogonal and composable. Adding a quantization format costs one device decode function plus one traits row plus one table entry, and no new kernel; we state this as a constant-cost extension proposition and report all 23 native-resident formats decoding bit-exact (`nbad` = 0) against the CPU reference. The performance claims are measured head-to-head against `llama.cpp` (which is the engine inside Ollama, so these are also the Ollama-class comparison) on the *same* GPU, same model, same quantization, under a device-map-controlled protocol. On the AMD Ryzen AI Max+ 395 “Strix Halo” APU (`gfx1151`) Hanzo *beats* `llama.cpp` on prefill for every model tested against the HIP backend (1.02–1.14× dense, 1.13× on a 30B mixture-of-experts) and against the Vulkan backend on the 4B/8B/30B models (1.14–1.26×); on decode it beats `llama.cpp`-HIP on the 30B-A3B mixture-of-experts (1.20×) and on the IQ3_XXS i-quant (1.13×), and reaches HIP parity on the dense workhorses (0.90–0.96×). On the NVIDIA GB10 “Grace Blackwell” it beats `llama.cpp`-CUDA on small-model decode (1.10× on 0.6B via a captured decode graph), reaches decode parity on 8B, and leads CUDA mixture-of-experts prefill (1.87–2.1× via an expert-grouped GEMM). We are equally explicit about the active frontier, marked in-flight with the lever named rather than dressed up: decode versus `llama.cpp`-Vulkan on the APU (0.85–0.96×, an attention-epilogue fusion); CUDA *dense*-attention prefill, where a Blackwell-tuned flash-attention kernel is now built, numerically stable on `sm_121`, and shipped in the CUDA images, lifting dense prefill +42% (`pp1024` 1425.5 → 2030.9 tok/s) and closing the gap to `llama.cpp` from 0.55× to 0.77–0.84× yet leaving dense-attention prefill the one axis Hanzo still trails; and the 35B hybrid gated-delta-net prefill on CUDA, where a warp-per-column recurrence now *beats* `llama.cpp`’s GDN kernel (0.93 vs 1.04 ms/layer, total prefill 354 → 1166 tok/s) with a `bf16` tensor-core GEMM the next lever for the projections and experts. Every number here is measured on named silicon at a named engine version; where a path is a projection or a microbenchmark it is labelled. The companion paper [1] carries the formal roofline analysis and the kernel-level campaign; this paper is the cross-backend, cross-modality overview, and its honesty is the point: the data carries the story without embellishment.

1 Introduction

Most inference stacks are a federation: a CUDA engine, a separate Metal port, a third thing for AMD, a fourth for CPU, and per-format quantization kernels copied and specialized until the matrix of (backend × format × dense/MoE) is unmaintainable. Hanzo takes the opposite position, the one our engineering culture states as a law: one and only one way to do everything, composable and orthogonal. The same Hanzo Engine build runs CUDA, ROCm, Metal, Vulkan, and CPU behind one model loader; the same Hanzo ML quantization core serves the full 1-bit-to-8-bit GGUF zoo across all of them; the same pipelines drive text, vision, audio, speech, image generation, and embeddings. There is no per-vendor inference stack to maintain and no rewrite to move a workload from a Mac to a Grace-Blackwell box to an APU mini-PC.

This paper makes two kinds of claims, and keeps them separate. The first is *breadth*, which is a matter of what the code does: the backend set, the modality set, the architecture set, and the quant zoo (Sections 2 and 3). The second is *performance*, which is a matter of measurement: same-GPU, same-quant, same-metric head-to-head numbers against `llama.cpp` (Section 5). We treat `llama.cpp` [2, 3] as the baseline throughout because it is the de-facto standard for edge and commodity LLM inference, and because Ollama [4] wraps `llama.cpp/ggml`: our same-GPU

`llama.cpp` numbers are therefore the Ollama-class comparison, stated once here so it need not be repeated. We do not benchmark against vLLM [8], whose home is batched datacenter serving; this paper is single-stream throughput on commodity unified-memory hardware.

The campaign behind these numbers has an ethos worth stating up front, because it determines how to read the results: **measured honesty**. The companion paper [1] documents negative results as first-class citizens, a fully-debugged HIP-graph effort that delivered zero speedup, a flash-decode kernel built and reverted, a lossless speculative decoder that is a net slowdown on this hardware. We continue that discipline here. Where Hanzo leads we show the data and let it carry; where it trails we say so, quantify the gap, and name the one lever that closes it. A paper of real measured wins plus honest frontier framing is more credible, and more useful to the people deciding what to deploy, than a paper that claims to win everything.

Summary of measured standing. Against `llama.cpp` on the same silicon:

- **Prefill, AMD gfx1151:** Hanzo *wins* on every model versus the HIP backend (1.02–1.14× dense; 1.13× on the 30B-A3B MoE) and versus the Vulkan backend on the 4B/8B/30B models (1.14–1.26×).
- **Decode, AMD gfx1151:** Hanzo *wins* versus HIP on the 30B-A3B MoE (1.20×) and on IQ3_XXS (1.13×); reaches HIP parity on dense 4B/8B (0.90–0.96×); trails the Vulkan backend (0.85–0.96×), the named frontier.
- **Decode, NVIDIA GB10:** Hanzo *wins* versus CUDA on 0.6B (1.10×, via the captured decode graph) and reaches parity on 8B.
- **Prefill, NVIDIA GB10:** Hanzo *leads* CUDA MoE prefill (1.87–2.1× via the expert-grouped GEMM) and the GDN recurrence kernel (0.93 vs 1.04 ms/layer; 35B prefill 354 → 1166 tok/s). *Dense*-attention prefill is the lone trailing axis: flash-attention is now shipped and lifts it +42% (0.55× → 0.77–0.84× `llama.cpp`), but it still trails at $\approx 0.8\times$.
- **Coverage:** the only portable multi-backend stack carrying all 22 GGUF formats plus MXFP4 native-resident through a single compute core, every one bit-exact; full multimodality including a unified omni model (text, vision, audio, speech).
- **In-flight (lever named):** CUDA dense prefill residual (0.77–0.84× with flash-attention shipped; the remaining $\approx 20\%$ is the `f32↔bf16` cast chain plus MMQ GEMM efficiency); the 35B hybrid total prefill (the GDN recurrence kernel now leads; `bf16` tensor-core GEMM next for the projections and experts); APU decode versus Vulkan (0.96×; attention-epilogue fusion).

2 Hanzo ML: The Unified Quantization Compute Core

Hanzo ML is the tensor and kernel layer: a Rust framework with a device abstraction over five backends and a quantization core that is the technical heart of the stack. Its design commitment, a standing user mandate, is to support quantization from 1-bit to full precision with *no per-quant kernels*: one quant-agnostic GPU core per stage, one decode function per format, reused across decode, prefill, and mixture-of-experts.

2.1 The add-a-quant contract and the orthogonal axes

The core is templated on a weight-format type parameter `WTYPE` and factors the problem into orthogonal axes that compose rather than multiply:

- **Weight format** (`WTYPE`): the GGUF block layout and its dequantization, isolated in one device function `qdec<WTYPE>::partial` and one traits row `qdw_traits<WTYPE>` (block bytes, elements, symmetry, sub-unit count).
- **Activation dtype** (`Act`): `f16`, `bf16`, or `f32`, selected at the entry point, so the residual stream can stay in its native dtype without a cast chain.
- **Dense versus MoE**: a single `bool` MoE template parameter; under `if constexpr(MoE)` the same core adds only an expert-grouped gather/scatter, and the dense path codegens byte-identical.
- **Scalar versus dp4a**: an `int8` dot-product decode path (`qdp4a<WTYPE>`) sits beside the scalar path, trait-selected per type, with the scalar core as the always-correct fallback.

These axes are realized as three matched cores: `qmatvec_core<WTYPE,Act>` for decode (with its MoE twin and its `dp4a` twin), `qmmq_core<WTYPE,MoE,NWAVE_M>` for the `int8` matrix-core prefill GEMM, and fused service kernels (`moe_route` for softmax/top-*k*/normalize, `moe_combine` for the expert reduction, `rope_norm_positions` for fused qk-RMSNorm-into-RoPE). The decompilation is deliberate: where three sibling efforts had each invented a prefill-capability predicate, the merged core has exactly one, `RocmQuantType::qmmq_capable`, read at every site. The format identifier is a *value* (`RocmQuantType`) qualified by namespace, not a place: dispatch reads the (type, activation) pair off the enum and selects the single core’s entry point. We state the resulting structural property precisely.

Proposition 1 (Constant-cost format extension). *Let the unified core be parameterized by `WTYPE`. Adding support for a new quantization format `F` to the decode, prefill, and MoE paths requires exactly (i) one device decode function `qdec<F>::partial` (and, optionally, one `qdp4a<F>` for the `int8` path), (ii) one traits row `qdw_traits<F>`, and (iii) one dispatch-table entry `RocmQuantType::F`. No new kernel, launcher, or dispatch branch is introduced; the matvec, GEMM, MoE gather/scatter, and warp-reduction code are shared and unchanged. Hence the per-format marginal source is $O(1)$ and independent of the number of already-supported formats.*

Proof sketch. By construction of the templated cores. The accumulation, reduction, and (for prefill) the 128×128 `iu8`-WMMA tiling loop are written once and instantiated per `WTYPE`; the only per-type code they call is `qdec<WTYPE>/qdp4a<WTYPE>` and the values in `qdw_traits<WTYPE>`. Dispatch is by symbolic kernel name keyed on `RocmQuantType`, so a new variant adds one match arm, not a branch in the hot loop. The empirical witness is that the six originally-wired formats and the sixteen added in one wave produce no new kernel and pass the same numeric oracle; the refactor of the prefill GEMM onto the template produced byte-identical disassembly. The formal bit-exactness of each instantiation is Lemma 1 of the companion paper [1]. $\square$

2.2 The complete quant zoo

The contract of Proposition 1 is realized to completion: **all 22 GGUF quantization formats plus MXFP4 (23 types total) are wired native-resident**, every one bit-exact against the CPU `to_float` reference (`nbad` = 0 across 233 oracle assertions, zero regression on the pre-existing set). The types span every structural class (Listing 1).

```

1 // k-quants:   Q2_K Q3_K Q4_K Q5_K Q6_K           (256-elem super-blocks)
2 // legacy:     Q4_0 Q4_1 Q5_0 Q5_1 Q8_0 Q8_1       (32-elem blocks)
3 // i-quants:   IQ1_S IQ1_M IQ2_XXS IQ2_XS IQ2_S
4 //             IQ3_XXS IQ3_S IQ4_NL IQ4_XS         (codebook / grid LUT)
5 // ternary:    TQ1_0 TQ2_0                         (base-3 packed)
6 // fp4:        MXFP4                               (E8M0 scale + FP4 codebook, gpt-oss)

```

Listing 1: The 23 native-resident formats, one `qdec` body and one traits row each.

One `qmmq_capable` predicate partitions the set: 11 formats carry an int8-WMMA prefill kernel; the rest dequantize to `f16` for prefill while decoding native. The whole zoo collapses onto *two* scalar accumulation shapes folded inside each decode (symmetric $\text{scale} \cdot q$, asymmetric $d \cdot \text{sc} \cdot q - d_{\min} \cdot m$), so the cores themselves are shape-agnostic. The honest distinction from `llama.cpp` is not coverage; `llama.cpp` defines the GGUF formats and supports them all. It is the *architecture*: Hanzo serves the entire zoo through one `WTYPE`-templated core family, the same cores driving decode, prefill, and MoE across five backends, with the one-line add-a-quant contract of Proposition 1; `llama.cpp` uses per-format front-ends and is primarily a single vendor’s kernel set per backend. Among portable, pure-Rust, multi-backend engines, Hanzo ML is the only one carrying the full zoo native-resident through a single compute core.

2.3 Five backends, one loader

The `Device` abstraction exposes `Cpu`, `Cuda`, `Metal`, `Rocm`, and `Vulkan`, each with a full backend implementation behind one storage trait. A model is loaded once and runs on whichever backend is present; the cross-backend decode parity reported in Section 5 is itself evidence that the unified core is not over-fit to one vendor. What is *ported* and what is *novel* is stated plainly, as in the companion work: the `mul_mat_q` integer-matrix-core prefill design and the GGUF block formats follow `llama.cpp/ggml`; the unification (one `WTYPE`-templated core spanning matvec, GEMM, and MoE with a one-line add-a-quant contract) is Hanzo’s.

2.4 Decode fusions and the bandwidth wall

The optimization program is organized by a single roofline dichotomy, which we state as the operating principle of the whole stack and whose formal ceiling is proved in the companion [1, 6].

Proposition 2 (Two-regime roofline). *Single-stream decode (batch 1, one new token) streams every weight once per token against a single activation row; it is therefore memory-bandwidth bound, with throughput at most B/W tokens per second for sustained bandwidth B and resident footprint W . Prefill (a multi-token prompt) multiplies a weight matrix by an activation matrix; it is compute bound on the matrix or tensor cores. Consequently decode optimization must reduce bytes streamed or kernel/launch overhead, and prefill optimization must improve matrix-core scheduling; a technique aimed at the wrong regime cannot move the bottleneck.*

This dichotomy explains every decode result. On `gfx1151` the per-expert decode matvec already sustains 88–99% of the 212 GB/s memory roofline (the `Q6_K` matvec does *more* arithmetic yet reaches *higher* bandwidth than `Q4_K`, which is impossible under a compute bound), so the matvec is finished and the remaining decode lever is *kernel count*, not kernel speed. Acting on that diagnosis, a sequence of fusions, each bit-exact and each a true elimination of work rather than a mere launch merge, moved end-to-end decode: a fused `moe_route` kernel collapsing the six-launch-per-layer routing chain to one (+13% on the 30B-A3B MoE); a fused gate/up shared activation-quantize

plus a fused residual-add RMSNorm (+2.8%); a fused qk-RMSNorm-into-RoPE kernel (+2.8%). The i-quants, which decoded through a scalar codebook lookup and were pathologically compute-bound, were given a `dp4a` grid-dot path that lifted `IQ3.XXS` decode $2.27\times$ (from 30.8 to ≈ 68 tok/s), enough to *pass* `llama.cpp`-HIP on that format. These are the wins behind the decode numbers in Section 5.

2.5 Honest negatives, as first-class results

The same measurement discipline ruled out a set of plausible levers, and recording them is part of the contribution because each one would otherwise be re-attempted.

- **RDNA3.5 iu8 WMMA equals f16 WMMA ($1.00\times$).** A register-only microbench measured 57.1 TOP/s int8 versus 56.8 TFLOP/s f16; unlike CDNA’s $2\times$ int8 advantage, RDNA3.5 has none. So on this APU quantization is a decode-side bandwidth/footprint play and *never* a prefill-side compute play. A hardware limit, not a tuning gap.
- **HIP-graph decode is correct but inert when already de-launch-bounded.** Capturing and replaying the per-token decode graph delivered exactly zero speedup once the fusion work had removed the launches it was built to collapse, reproduced at two scales (8B and a 30B MoE) and two platforms. Graphs pay only where launch-bound (small models, and the MoE routing chain before it was fused), which is exactly where we now enable them.
- **Matvec rewrites are flat at the wall.** f32-native activation streaming, a fused router GEMV, and dense lane-coalescing are each bit-exact yet move device-map-controlled decode by $\leq 1.3\%$, because they optimize a kernel already at 88–99% of bandwidth. Folding the activation quantize *into* the matvec was bit-exact but -22% : it injects redundant per-block work into a bandwidth-bound kernel. Reverted.
- **Lossless speculative decode is a net slowdown here.** A classic draft-plus-target decoder, added as a second proposer on the existing trait, is provably lossless at temperature 0 (greedy output token-identical with and without a draft) yet measures $0.4\text{--}0.63\times$ on this APU with stock small drafts: acceptance is only 33–40%, the verify is charged at the prefill rate, and a 3B-active MoE has no per-token headroom to amortize. Shipped, proven, and recommended against here; the multi-token-prediction path on the same trait is the viable variant.

The pattern is consistent: we profile the production kernel, attribute every millisecond, optimize only what the profiler proves is the bottleneck, and report the disproven ideas as valid negative results.

3 Hanzo Engine: Architectures, Modalities, Pipelines

Hanzo Engine is the inference server and model layer on top of Hanzo ML: the model implementations, the pipelines (text, multimodal, speech, diffusion, embedding), the OpenAI- and Anthropic-compatible HTTP surface, the MCP and Python and Rust SDKs, PagedAttention, and the decode-graph and speculative machinery. Its breadth is the second half of the thesis.

3.1 Architecture breadth

The engine ships 32 text architectures and 27 vision modules behind one loader. The set spans three structural families that matter for inference, summarized in Table 1.

Table 1: Architecture and modality breadth (representative, not exhaustive).

Family / modality	Representative models
Dense LLM	Llama, Mistral, Qwen3, Gemma3, Phi3, Granite, Starcoder2, SmolLM3
MoE LLM	Qwen3-MoE, Qwen3.5-MoE, Mixtral, Phi3.5-MoE, GLM4-MoE, GPT-OSS, DeepSeek-V
Hybrid GDN+MoE	Qwen3-Next (gated-delta-net linear attn + full attn + MoE)
Vision-language	Qwen2.5-VL, Qwen3-VL(-MoE), Llama4, Gemma3/3n, Idefics2/3, LLaVA, Mistral3, Phi
Audio in	Voxtral, Conformer, Qwen3-ASR, MiniCPM-O
Speech out (TTS)	Dia, Qwen3-TTS
Omni	Qwen3-Omni (text + vision + audio + speech)
Image generation	FLUX, Qwen-Image (plus MuseTalk lip-sync)
Embeddings	Qwen3-Embedding, EmbeddingGemma

- **Dense transformers:** Llama, Mistral, Qwen2/Qwen3, Gemma/Gemma2/Gemma3, Phi2/Phi3, Granite, Starcoder2, SmolLM3.
- **Mixture-of-experts:** Qwen3-MoE (the 30B-A3B workhorse), Qwen3.5-MoE, Mixtral, Phi3.5-MoE, GLM4-MoE, GPT-OSS (MXFP4 experts), and DeepSeek-V2/V3 (multi-head latent attention plus MoE).
- **Hybrid linear-attention plus MoE:** Qwen3-Next, which interleaves gated-delta-net (GDN) linear-attention layers [9] with a full-attention layer every fourth layer and an MoE feed-forward. This is the 35B-A3B hybrid benchmarked in Section 5.3; it exercises a recurrence (the chunked gated delta rule) and a causal conv1d in addition to the usual attention and MoE kernels.

3.2 Modality breadth

The pipeline layer exposes six model categories: text, vision, audio, speech, diffusion (image generation), and embeddings. Vision-language support is broad (Qwen2.5-VL, the Qwen3-VL family, Llama4, the Gemma3 line, Idefics2/3, LLaVA, Mistral3, Phi4, with CLIP/SigLIP encoders); audio input runs through Voxtral, a Conformer, and Qwen3-ASR; speech synthesis runs through Dia and Qwen3-TTS; image generation runs through FLUX and Qwen-Image. The capstone is **Qwen3-Omni**, a single model that ingests text, images, and audio and emits text and speech, served by the same engine and the same quantized cores as the text models. This is the multimodal breadth the stack claims: not a bag of separate tools, but one engine and one quantization core spanning the modalities.

3.3 Pipelines: paged attention, decode graphs, speculative decode

Three pieces of pipeline machinery are worth naming because they recur in the results. **PagedAttention** [8] is implemented on every backend (the ROCm port was a keystone built during the campaign and is the prerequisite for the speculative and prefix-cache paths). **Decode graphs** capture the steady-state single-token decode and replay it as one GPU graph instead of relaunching hundreds of kernels per token; they are wired for safetensors and GGUF on both CUDA and ROCm, gated by an eligibility invariant (only architectures whose decode RoPE reads a stable device position buffer are graph-safe, which excludes host-offset and per-forward mRoPE variants that would otherwise freeze the position at capture and emit garbage on replay). The CUDA GGUF

Table 2: Hardware. All three are commodity unified-memory boxes (no discrete VRAM tier).

Box	Accelerator	Memory / stack
AMD Strix Halo	Radeon 8060S iGPU, <code>gfx1151</code> (RDNA3.5), 40 CU @ 2.9 GHz	62 GB unified LPDDR5X, $\approx$
NVIDIA GB10	Grace-Blackwell, <code>sm_121</code> , integrated GPU	122 GB unified LPDDR5X,
Apple M4 Max	M4 Max GPU, Metal	137 GB unified; <code>llama.cpp</code>

decode graph is the lever behind the small-model CUDA decode win in Section 5.3. **Speculative decoding** is factored behind one `SpeculativeProposer` trait with two implementors, draft-model and multi-token-prediction, composed onto the same verifier; the draft path is proved lossless at temperature 0 (Section 2.5).

4 Method

Every performance number in this paper is measured on named silicon (Table 2), at a named engine version, against `llama.cpp` on the *same* GPU, same model, same quantization, quiet GPU. The protocol controls the confounds that make unified-memory benchmarking treacherous.

- **Same-GPU, same-quant, same-metric.** Prefill is 512- or 1024-token prompt throughput (`pp512/pp1024`); decode is single-stream token generation at shallow KV depth. For `llama.cpp` these are `llama-bench`’s `pp/tg`; for Hanzo they are the `hanzo bench` prefill and decode at matched lengths. We compare against the `llama.cpp` HIP *and* Vulkan backends on AMD, because `llama.cpp` itself decodes faster on Vulkan than HIP on this APU, and comparing only against its slower backend would flatter us.
- **Device-map control.** On a unified-memory box the auto device-mapper can misread shared RAM as per-layer VRAM and silently offload roughly a third of the layers to the CPU, which collapses decode (prefill, being compute-bound, survives) and manufactures illusory speedups. We force all layers resident on the accelerator for both engines (`-ngl 99` for `llama.cpp`; `-n 0:NLAYERS` for Hanzo). An uncontrolled A/B against a mis-mapped baseline once produced an apparent “+28%” that was entirely the device fix; held constant, that same change was flat.
- **Thermal control.** The APU’s warm runs swing $\approx 20\%$, so sequential comparison is untrustworthy; A/B comparisons are thermally interleaved and ratios are taken per round.
- **Correctness gating.** Every kernel change is gated bit-exact (`nbad = 0`) against a CPU numeric oracle before any timing is trusted, and output coherence is verified.

5 Results

5.1 AMD gfx1151: prefill won, decode at HIP parity-or-better

Table 3 is the headline same-GPU comparison on the Strix Halo APU: Hanzo (ROCm [10]) against `llama.cpp` on both its HIP and its Vulkan backend, across three dense Qwen3 [5] models and the 30B-A3B mixture-of-experts. Two facts dominate.

Table 3: AMD `gfx1151`, Hanzo ROCm vs `llama.cpp` HIP and Vulkan on the *same* GPU (`pp1024/tg128`; the 30B-A3B is Q4_K_M, decode on the production HIP-graph path). Throughput in tok/s; ratio is Hanzo / `llama.cpp`. Bold marks a Hanzo lead.

Model	Prefill (tok/s)			Decode (tok/s)		
	Hanzo	vs HIP	vs Vk	Hanzo	vs HIP	vs Vk
Qwen3-0.6B-Q8_0 (dense)	6966	1.14 $\times$	0.96 $\times$	128	0.69 $\times$	0.60 $\times$
Qwen3-4B-Q8_0 (dense)	1750	1.02 $\times$	1.19 $\times$	58.2	0.96 $\times$	0.85 $\times$
Qwen3-8B-Q8_0 (dense)	1066	1.05 $\times$	1.14 $\times$	34.6	0.90 $\times$	0.87 $\times$
Qwen3-30B-A3B (MoE)	1209	1.13 $\times$	1.26 $\times$	79.5	1.20 $\times$	0.96 $\times$

Table 4: AMD `gfx1151`, Qwen3-30B-A3B decode by quantization format (cross-quant @d4 pass). Throughput in tok/s on the same GPU; ratio is Hanzo / `llama.cpp`.

Format	bpw	Hanzo	llama-HIP	llama-Vk	vs HIP
Q4_K	4.5	64.7	66	83	0.98 $\times$
Q2_K	2.6	75.0	79	92	0.95 $\times$
IQ3_XXS	3.06	65.5	58	91	1.13 $\times$

First, **Hanzo wins prefill outright**. Against the HIP backend it is faster on every model (1.02–1.14 $\times$ dense, 1.13 $\times$ on the 30B MoE); against the heavily community-tuned Vulkan backend it wins on the 4B, 8B, and 30B models (1.14–1.26 $\times$) and is within 4% on the tiny 0.6B. The MoE prefill win is the headline kernel result of the campaign: a fused expert-grouped WMMA GEMM plus a fused expert-combine took MoE prefill from 0.34 $\times$ to a 1.13 $\times$ *lead* over `llama.cpp`-HIP, entirely through kernels the engine owns.

Second, **decode is at HIP parity-or-better**. Hanzo beats `llama.cpp`-HIP on the 30B-A3B MoE (1.20 $\times$, the production path with HIP-graphs default-on and the full decode-fusion campaign) and reaches HIP parity on the 4B/8B workhorses (0.90–0.96 $\times$). It trails HIP only on the tiny 0.6B (0.69 $\times$), where decode is dispatch-overhead-bound and the fixed per-token launch cost dominates a small weight stream. Against the Vulkan backend, decode trails by 0.85–0.96 $\times$: this is the active frontier (Section 6), and the gap is structural, Vulkan’s sustained occupancy and fused attention epilogue, not a slower matvec (ours is at the bandwidth wall).

5.2 The per-quant decode ladder

Because the quant zoo is the architectural centerpiece, Table 4 shows decode across three formats on the 30B-A3B MoE in one controlled cross-quant pass (the @d4 bench harness at an intermediate engine version, used here for the relative comparison; the production decode-fusion path of Table 3 raises the absolute Q4_K figure further, as noted). Two readings matter. The **k-quant bandwidth ladder works**: lower bits decode faster (Q4_K $\rightarrow$ Q3_K $\rightarrow$ Q2_K), consistent with Proposition 2, and Hanzo is at HIP parity across the k-quant. And **the i-quant dp4a path is a genuine win**: after vectorizing the codebook decode, IQ3_XXS decode rose 2.27 $\times$ and now *beats* `llama.cpp`-HIP (1.13 $\times$). The residual gap to the Vulkan backend is uniform across formats and is the same structural attention-epilogue gap as in Table 3, not a per-format kernel deficit.

Table 5: NVIDIA GB10, Hanzo CUDA vs `llama.cpp`-CUDA on the *same* GPU and GGUF. Prefill/decode are Hanzo / `llama.cpp` ratios; the 8B prefill cell is no-FA→flash-attn (full prompt-length sweep in Table 6), and the 35B prefill cell is throughput trajectory in tok/s (its GDN recurrence kernel leads `llama.cpp` at 0.93 vs 1.04 ms/layer). Decode in tok/s where shown. Bold marks a Hanzo lead.

Model	Prefill	Decode	Status / lever
Qwen3-0.6B (dense)	0.33×	1.10× (247.5 vs 225.6)	decode-graph shipped; dense prefill vi
Qwen3-8B (dense)	0.55→0.80×	≈0.98×	flash-attn shipped (+42%); residual: c
Qwen3-Next-35B (GDN+MoE)	354 → 1166	runs coherent	GDN kernel leads (0.93 vs 1.04 ms/la

5.3 NVIDIA GB10: decode and MoE prefill won, dense prefill the frontier

On the Grace-Blackwell GB10, Table 5 compares Hanzo (CUDA) against `llama.cpp`-CUDA on the same GPU and GGUF. The decisive recent result is the **CUDA GGUF decode graph**: wiring the captured decode graph into the GGUF pipeline (it had been running eager, ≈ 780 kernel launches per token) lifted 0.6B decode from 175.6 to 247.5 tok/s (+41%), crossing `llama.cpp`-CUDA (225.6) to 1.10×, byte-identical to the eager output. On the 8B, decode is at parity ($\approx 0.98\times$): there the model is bandwidth-bound and the graph adds little, exactly as Proposition 2 predicts.

Prefill on CUDA splits cleanly by regime, and the split is the honest story. On the *sparse* and *recurrent* paths Hanzo leads: the expert-grouped MoE prefill GEMM ported to CUDA (the same kernel family that took AMD MoE prefill to a 1.13× lead, Section 5.1) delivers 1.87–2.1× MoE prefill over `llama.cpp`-CUDA, and on the 35B-A3B hybrid GDN+MoE a warp-per-column gated-delta-rule recurrence now *beats* `llama.cpp`’s GDN kernel (0.93 vs 1.04 ms/layer). That hybrid shows the full campaign trajectory: de-redundifying the chunked recurrence (eliminating $\approx 370\times$ redundant `exp` calls, $18.3 \rightarrow 4.0$ ms/layer), wiring `conv1d` to a CUDA kernel, fusing the gated RMSNorm, and finally the warp-per-column recurrence took its prefill from 354 to 1166 tok/s (a 3.3× gain over the floor), bit-exact to an `f64`-truth reference at $\approx 10^{-7}$ and coherent (the silent CUDA MoE routing-corruption bug that once panicked it is fixed and gated by a regression oracle). The residual on the 35B *total* prefill is the projection and expert GEMM; the named next lever is a `bf16` tensor-core GEMM there.

The one axis where Hanzo still trails is *dense*-attention prefill, and Section 5.4 reports it without inflation. Profiling attributed the GGUF dense-prefill gap (0.33–0.55×) to un-fused attention (softmax plus a materialized score tensor) and `f32↔bf16` casts, with the actual MMQ GEMM only a small slice. The named lever, a Blackwell-tuned flash-attention [7] kernel, is now *built and shipped* (Table 6): it lifts 8B dense prefill +42% and closes the gap to 0.77–0.84×, but does not erase it.

5.4 Flash-attention on Blackwell: the dense-prefill lever, shipped

Dense-attention prefill is the single axis on which Hanzo trails `llama.cpp`, and the named lever to close it was a Blackwell-tuned flash-attention [7] kernel. That kernel now exists, is correct, and ships. We report the result without inflation (Table 6): on the GB10 (`sm_121`, CUDA 13.0), same Qwen3-8B Q4_K_M GGUF as `llama.cpp`-CUDA, flash-attention lifts dense prefill +42% at `pp1024` ($1425.5 \rightarrow 2030.9$ tok/s) and holds the gain across context lengths. Crossing the kernel off as built is itself a result: the CUTLASS-for-Blackwell attention path compiles and runs *numerically stable* on `sm_121` (coherent generation, no overflow), clearing the real risk that the new architecture would demand a different kernel.

Table 6: NVIDIA GB10 (sm_121, CUDA 13.0), Qwen3-8B Q4_K_M GGUF dense prefill: Hanzo CUDA (without and with flash-attention) vs llama.cpp-CUDA on the *same* GPU and GGUF. Throughput in tok/s (higher is better); “FA vs llama” is Hanzo+flash-attn / llama.cpp. Flash-attention lifts dense prefill +42% and closes the gap to 0.77–0.84 $\times$, but dense prefill remains the one axis Hanzo trails.

Prompt	Hanzo no-FA	Hanzo +FA	llama.cpp	FA vs llama
pp1024	1425.5	2030.9	2542.4	0.80 $\times$
pp2048	—	2126.4	2537.1	0.84 $\times$
pp4096	—	1902.2	2465.5	0.77 $\times$

Table 7: Cross-backend, Qwen3-8B-Q8_0, single GPU (companion measurement pass [1]). Decode parity on all three; the hardware optimum flips between prefill and decode.

Backend	Prefill (tok/s)			Decode (tok/s)		
	llama	Hanzo	ratio	llama	Hanzo	ratio
AMD Strix Halo (ROCm)	1176	1012	0.86 $\times$	25.0	24.5	0.98 $\times$
NVIDIA GB10 (CUDA)	2155	1630	0.76 $\times$	19.6	20.3	1.04 $\times$
Apple M4 Max (Metal)	873	792	0.91 $\times$	50.4	49.9	0.99 $\times$

The honest reading is the point. Flash-attention closes the dense-prefill gap from 0.55 $\times$ (no-FA) to **0.77–0.84 $\times$ llama.cpp**, a large improvement, *but dense-attention prefill remains the one axis where Hanzo still trails*, at $\approx 0.8\times$. We do not claim a dense-prefill win. The residual $\approx 16\text{--}23\%$ is the part of the prefill path that flash-attention does not touch: the `f32 $\leftrightarrow$ bf16` cast chain around the quantized GEMM (measured $\approx 20\%$) and MMQ GEMM efficiency. Those are the next two levers, and until they land this axis is stated as trailing, not won.

The win is shipped, not a branch. Flash-attention is in the CUDA container images: `Dockerfile.cuda` builds `WITH_FEATURES="cuda,cudnn,flash-attn"` on a CUDA 13.0.3 base, and the build matrix now includes Blackwell `sm_120/sm_121`, so GB10 and RTX-5090-class hardware get first-class CUDA images and the +42% reaches users rather than sitting in a local build.

5.5 Cross-backend: decode parity everywhere, and the box flips with the workload

A separate measurement pass [1] fixed one model (Qwen3-8B-Q8_0) and compared Hanzo to llama.cpp across AMD, NVIDIA, and Apple (Table 7). Two cross-cutting facts fall out. First, **Hanzo reaches decode parity with llama.cpp on all three backends** (0.98 $\times$ /1.04 $\times$ /0.99 $\times$ on AMD/NVIDIA/Apple), evidence that the unified core streams at the hardware ceiling regardless of vendor; prefill trails by a backend-dependent 0.76–0.91 $\times$ on that older pass (the AMD prefill has since become a lead, Table 3). Second, and orthogonal to the engine, **the fastest commodity box depends on the workload**: NVIDIA GB10 wins prefill (compute-bound; Blackwell tensor cores), Apple M4 Max wins decode by roughly 2.5 $\times$ (bandwidth-bound; widest unified memory), and the GB10, the strongest prefill box, is the slowest at decode. A RAG or document-ingest workload prefers the GB10; an interactive chat or long-generation workload prefers the M4 Max. The practical payoff of the unified stack is that the *same* build routes to whichever box the workload wants, with no per-vendor rewrite.

6 Where Hanzo Leads, and the Active Frontier

Where Hanzo leads (measured).

1. **Prefill on AMD gfx1151:** faster than `llama.cpp`-HIP on every model ($1.02\text{--}1.14\times$ dense, $1.13\times$ MoE) and faster than `llama.cpp`-Vulkan on 4B/8B/30B ($1.14\text{--}1.26\times$). The MoE-prefill lead is the campaign’s signature kernel win ($0.34\times \rightarrow 1.13\times$).
2. **Sparse and recurrent prefill on NVIDIA GB10 (CUDA):** the expert-grouped MoE prefill GEMM leads `llama.cpp`-CUDA ($1.87\text{--}2.1\times$), and the warp-per-column GDN recurrence kernel beats `llama.cpp`’s (0.93 vs 1.04 ms/layer; 35B prefill $354 \rightarrow 1166$ tok/s). Dense-attention prefill is the lone CUDA prefill axis still trailing (frontier item 2).
3. **Decode wins where the regime allows:** 30B-A3B MoE $1.20\times$ HIP; IQ3.XXS $1.13\times$ HIP; 0.6B $1.10\times$ CUDA via the decode graph; HIP and cross-backend decode parity on the dense workhorses.
4. **Completeness:** all 22 GGUF formats plus MXFP4 native-resident through one compute core, every one bit-exact; one engine across five backends; the full modality set including a unified omni model. This is the breadth no single-vendor or single-format stack matches.

The active frontier (in-flight, lever named). We mark these in-progress, not as losses, because each has a profiler-identified lever and measured forward motion.

1. **APU decode versus `llama.cpp`-Vulkan ($0.85\text{--}0.96\times$).** The matvec is at the bandwidth wall; the residual is Vulkan’s fused attention epilogue and sustained occupancy. The lever is fusing the whole attention epilogue (norm, RoPE, cache-append, cast-free), a multi-kernel rewrite, not a single op.
2. **CUDA dense-attention prefill ($0.77\text{--}0.84\times$, flash-attention shipped).** The Blackwell flash-attention kernel is built, numerically stable on `sm_121`, and in the CUDA images; it lifted dense prefill +42% (Table 6) and closed the gap from $0.55\times$, but this is the one axis Hanzo still trails. The residual $\approx 20\%$ is the `f32 $\leftrightarrow$ bf16` cast chain plus MMQ GEMM efficiency, the next two levers.
3. **35B hybrid *total* prefill on CUDA.** The GDN recurrence kernel itself now leads `llama.cpp` (0.93 vs 1.04 ms/layer) and total prefill rose $354 \rightarrow 1166$ tok/s; the remaining gap on the full hybrid is a `bf16` tensor-core GEMM for the projections and experts, the next item.
4. **Tiny-model decode dispatch overhead (0.6B HIP $0.69\times$).** Fixed per-token launch cost dominates a small weight stream; the decode graph (which already won on CUDA) and further op-fusion are the levers.

Two walls we could not move, we proved were silicon rather than software: the RDNA3.5 `iu8` WMMA rate (equal to `f16`, so quantization is a bandwidth play not a compute play on this APU) and the decode matvec’s 88–99% bandwidth saturation. Naming a ceiling as physics is itself a result; it tells the next engineer where *not* to spend effort.

7 Conclusion

The bottom line is deliberately unembellished. Hanzo ML and Hanzo Engine are one composable, multi-backend, multi-modal inference stack: one quantization core serving the full 1-bit-to-8-bit GGUF zoo bit-exact across CUDA, ROCm, Metal, Vulkan, and CPU, with a constant-cost add-a-quant contract (Proposition 1); one engine spanning text, vision, audio, speech, image generation, and embeddings, including a unified omni model. Measured against `llama.cpp` (and thus Ollama) on the same silicon, Hanzo *leads* on AMD prefill for every model, leads CUDA MoE prefill (1.87–2.1×) and the CUDA GDN recurrence kernel (0.93 vs 1.04 ms/layer), leads on decode where the regime allows (the 30B MoE at 1.20× HIP, the 0.6B at 1.10× CUDA, the IQ3.XXS i-quant at 1.13× HIP), and reaches decode parity across all three backends. The one axis it still trails is *dense*-attention prefill on CUDA: a Blackwell flash-attention kernel, now built and shipped, lifted it +42% and closed the gap to 0.77–0.84× `llama.cpp` but did not erase it, and the residual cast-and-GEMM levers are stated as engineering, not hidden. The remaining frontiers, APU decode versus Vulkan and the 35B hybrid total prefill, are likewise named with the lever and forward motion measured. The campaign’s ethos is the paper’s credibility: we profile the production path, gate every change bit-exact, control the device map and the thermals, and report the disproven ideas as first-class negative results. The data leads a lot; we let it carry, and we are precise where it does not.

References

- [1] Hanzo AI. *Native ROCm Inference on a Consumer RDNA3.5 APU: Reaching llama.cpp Decode Parity via a Unified 1-bit-to-Full Quantization Compute Core*. Hanzo AI white paper (companion), 2026.
- [2] G. Gerganov et al. *llama.cpp: LLM inference in C/C++*. 2023.
- [3] G. Gerganov et al. *ggml: Tensor library for machine learning*. 2023.
- [4] Ollama. *Ollama: Get up and running with large language models locally* (built on `llama.cpp/ggml`). 2023.
- [5] Qwen Team. *Qwen3 Technical Report*. Alibaba, 2025.
- [6] S. Williams, A. Waterman, and D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [7] T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*, 2022.
- [8] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023.
- [9] S. Yang et al. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. 2024.
- [10] AMD. *ROCm: Open Software Platform for GPU Compute*. Versions 7.1 and 7.13, 2026.